

HOWTO

CONTINUOUS PERFORMANCE ENGINEERING

Henrik Ingo

Fosdem 2026

POLL

Continuous Performance
Engineering
maturity levels

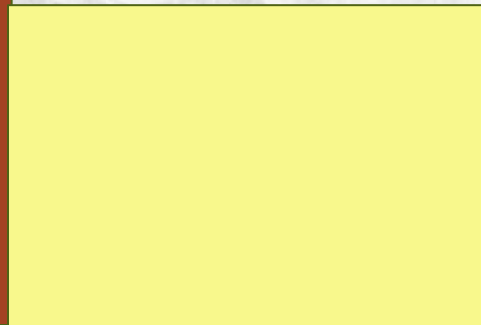
1. We don't really do benchmarking
2. Benchmarks yes, continuous no
3. Some benchmarks run in CI
...but we ignore the results
4. Benchmarks in CI,
Daily or more often,
Actionable results,
Regressions fixed within weeks

TYPICAL DISTRIBUTION OF ANSWERS (NORMAL DEVELOPERS)

50%

Simply don't care about
performance.
(Often rightly so!)

Google trends



TYPICAL DISTRIBUTION OF ANSWERS (NORMAL DEVELOPERS)

> 50%

< 50%

Simply don't care about
performance.
(Often rightly so!)

Benchmarking happens
Before release
One perf engineer
"Right Shift"

Benchmarks in CI
Noise 20% -100%
Alerts = distraction

Only a handful or
companies. 100?

CONTINUOUS INTEGRATION

CONTINUOUS TESTING

CONTINUOUS DEPLOYMENT

DEVOPS

LEFT SHIFTING

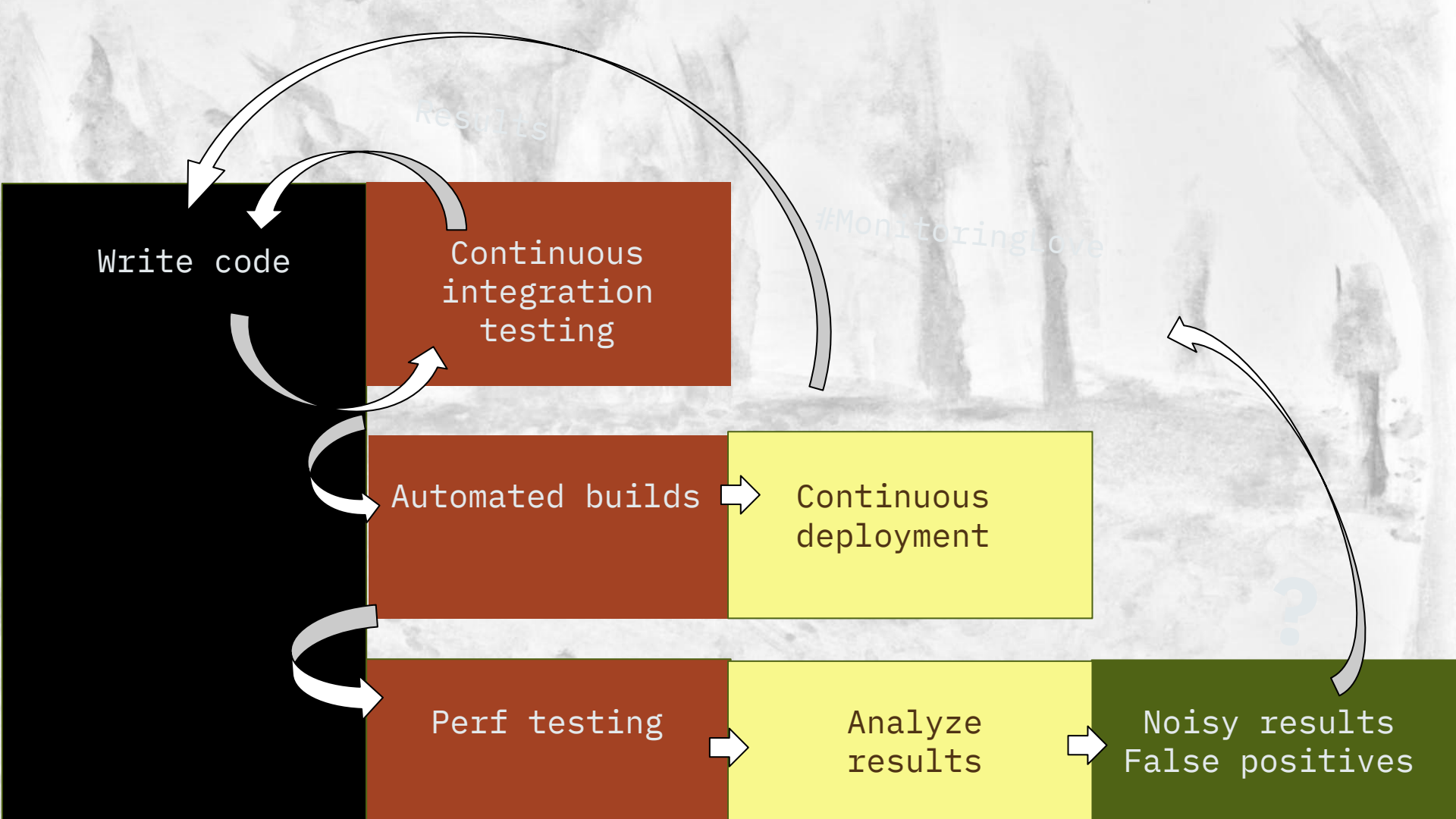
ITERATIONS

CONTINUOUS PERFORMANCE ENGINEERING?

Somewhere in that evolution, they forgot to bring the performance engineer



Velociraptor, the fastest dinosaur (wikipedia)



WHY WAS PERFORMANCE ENGINEERING LEFT IN 1999?

Deploying is necessary - benchmarking is optional

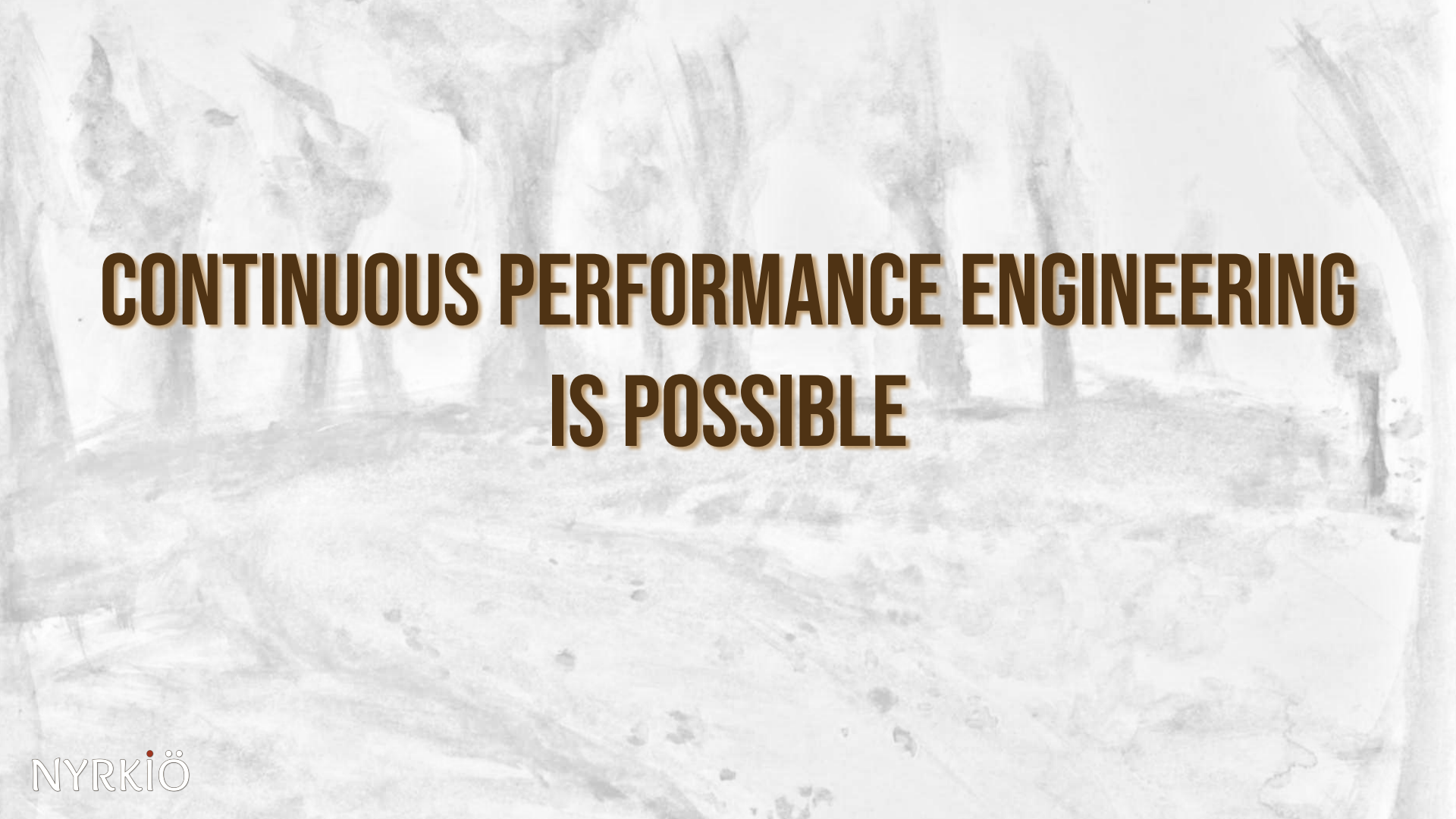
Performance Engineers busy fixing prod / customer problems

....and maybe enjoying it?

Math is hard

Relevant tuning is unintuitive, not well known

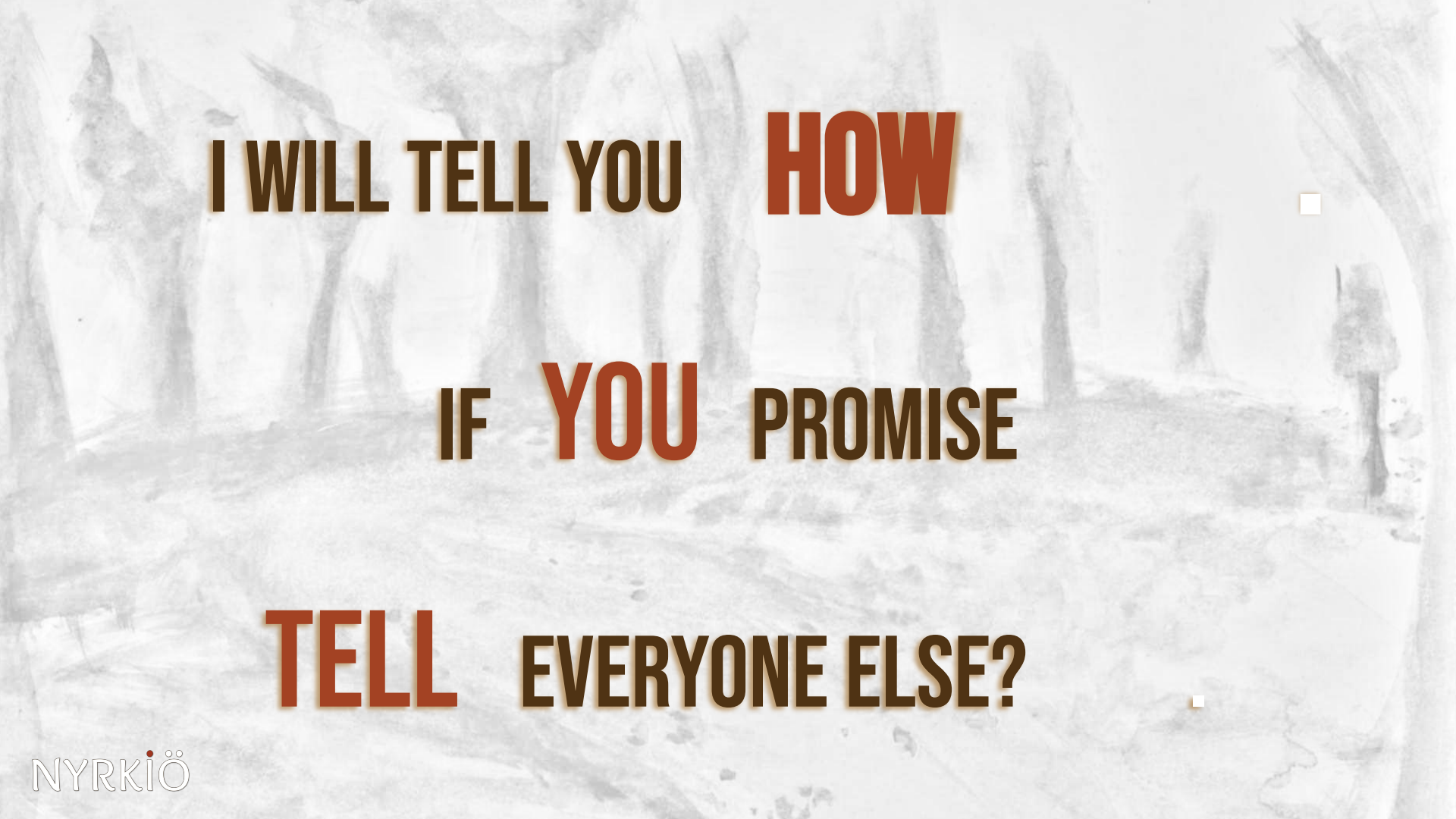
CONTINUOUS PERFORMANCE ENGINEERING



CONTINUOUS PERFORMANCE ENGINEERING IS POSSIBLE



CONTINUOUS PERFORMANCE ENGINEERING
IS POSSIBLE
IS ACHIEVABLE!



I WILL TELL YOU HOW

IF YOU PROMISE

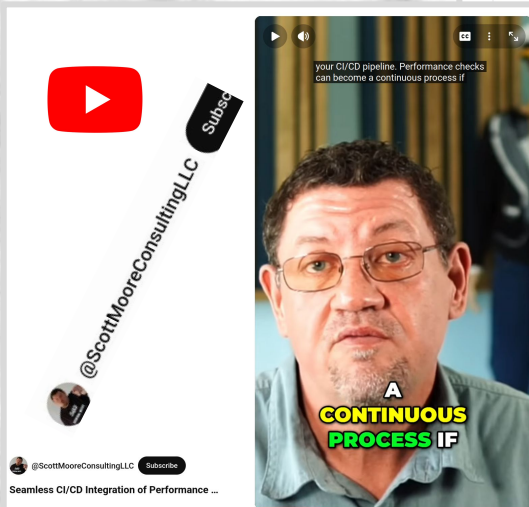
TELL EVERYONE ELSE?

POLL

The community is so new,
we don't even have consensus
for what to call it?

1. Continuous Performance Engineering
2. Continuous Benchmarking
3. DevPerfOps
4. Something else?
5. How should I know, I'm hearing about this for the first time...,.

WHO ELSE TALKS ABOUT THIS?



Continuous Benchmark Actions

GitHub Action for Continuous Benchmarking

release v1.20.7 CI passing codecov 89%

This repository provides a [GitHub Action](#) for continuous benchmarking. If your project has some benchmarking, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow runs. This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart of benchmark results are visualized on the GitHub pages of your project.



Apache Otava

Change Detection for Continuous Performance Engineering

Netherlands



Business Description:

Perfana delivers software performance engineering solutions to help you deliver high-performing software.

Based in: Amsterdam



NYRKiÖ

Dashboards

Product

Docs

About

{codspeed 🦘}

Documentation Guides

Discord

Get Started

Quickstart

Get Started

What is CodSpeed?

Integrated CI tools for software engineering to help you deliver the next delivery on performances.

ICPE

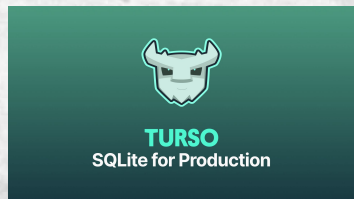
ACM/SPEC International Conference on Performance Engineering

WHO ELSE IS ALREADY DOING IT?

NETFLIX



DATASIX



TARANTOOL

NYRKIÖ



AGENDA

1. Benchmarking
Make it Continuous
2. Change Point Detection
Math & Science
3. Minimizing noise in the
Benchmarks
Assume nothing.
Measure everything.

BENCHMARKING PROCESS AND TOOLS

ON BENCHMARK TOOLS & DESIGN

Previous talk by Kemal Akkoyun & Augusto de Oliveira.

But in general...

- at this point each language has its own standard framework:
 - Java & JMH
 - Python & pytest-benchmark
 - etc...

Frameworks to run fully, end to end automated distributed benchmarks exist. Personally I believe we will see new innovation coming here. (k8s)

CORE PRINCIPLES (PART 1)

Sharing* is caring

Writing** is powerful

When writing, including at least one graph is mandatory

*) Benchmark results, findings

**) Or present at conferences

CORE PRINCIPLES (FOR CONTINUOUS BENCHMARK DESIGN)

You get what you measure

Basic scientific process: Hypothesize -> Configure -> Test -> Analyze -> GOTO 1

Only change one thing per iteration

CORE PRINCIPLES (FOR CONTINUOUS BENCHMARK DESIGN)

Benchmarks should be represent **realistic customer workloads**

However, that can take weeks, or months, just for a single benchmark.

Not agile!

One simple benchmark running in CI is more valuable than the perfect

Simple benchmarks are easier to analyze and reason about

For Continuous Benchmarking, having even one benchmark running in CI, allows you to start tracking it's history with graphs and change point detection!

GITHUB-ACTION-BENCHMARK

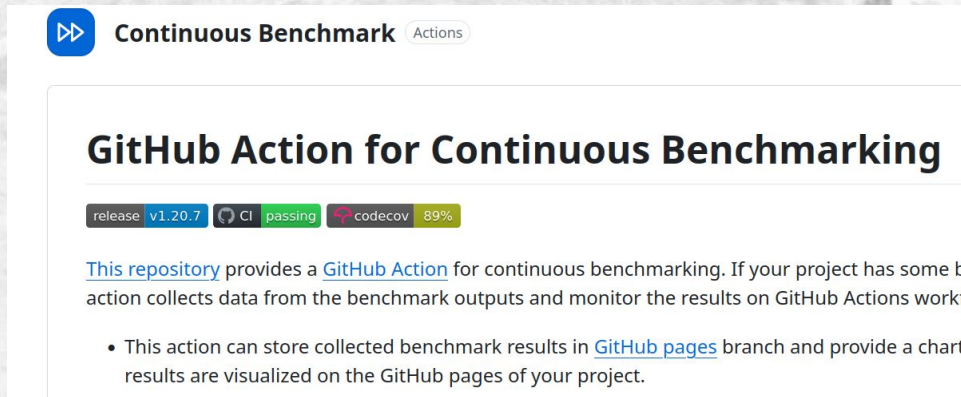
Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Threshold based alerts.

Default threshold = 100% (2x)



The screenshot shows the GitHub repository page for 'Continuous Benchmark' by 'Actions'. The repository name is 'Continuous Benchmark' and it is categorized under 'Actions'. The repository has a 'release' tag for 'v1.20.7', a 'CI' badge showing 'passing', and a 'codecov' badge showing '89%'. The repository description states: 'This repository provides a GitHub Action for continuous benchmarking. If your project has some benchmarking action, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow pages.' A bullet point indicates: 'This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart where the results are visualized on the GitHub pages of your project.'

GITHUB-ACTION-BENCHMARK

Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Threshold based alerts.

Default threshold = 100% (2x)

```
jobs:
  benchmark:
    name: Performance regression check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v4
        with:
          go-version: "stable"
      - name: Run benchmark with `go test -bench` and stores the output to a file
        run: go test -bench 'BenchmarkFib' | tee output.txt

      - name: Analyze benchmark results with Nyrkiö
        uses: nyrkiö/change-detection@v2
        with:
          tool: 'go'
          output-file-path: output.txt
```



CHANGE POINT DETECTION

A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.

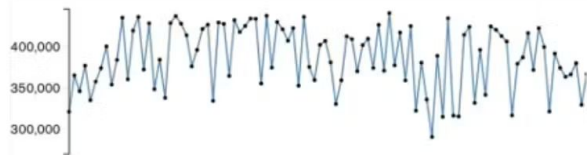
▼ insert_ttl-wiredTiger

c48f298

ops per sec:

313,407

Compare



▶ insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298

ops per sec:

492,207

Compare



▶ insert_vector_secondary_load_phase-wiredTiger

▶ insert_vector_secondary_overall-wiredTiger

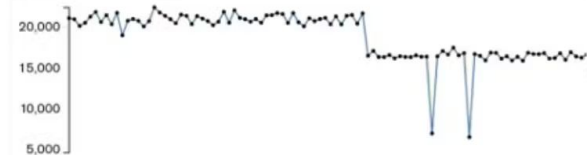
▶ word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298

ops per sec: 5,575

Compare



A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.

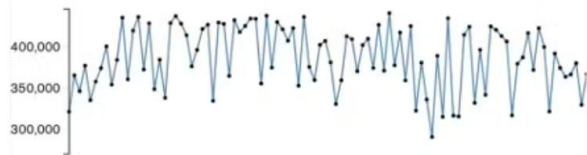
▼ insert_ttl-wiredTiger

c48f298

ops per sec:

313,407

Compare



40 % noise.
No regression!

▶ insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298

ops per sec:

492,207

Compare



17 %
No regression!

▶ insert_vector_secondary_load_phase-wiredTiger

▶ insert_vector_secondary_overall-wiredTiger

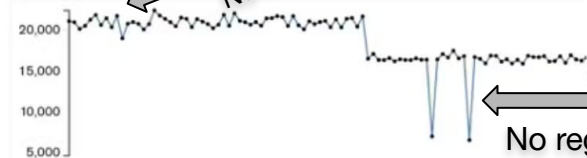
▶ word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298

ops per sec: 5,575

Compare



15 %
No regression!

70%
No regression!

A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.

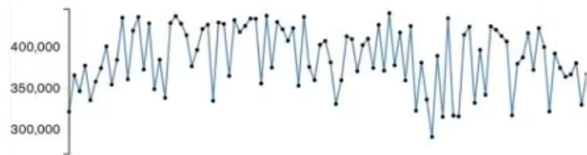
▼ insert_ttl-wiredTiger

c48f298

ops per sec:

313,407

Compare



40 % noise.
No regression!

▶ insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298

ops per sec:

492,207

Compare



17 %
No regression!

▶ insert_vector_secondary_load_phase-wiredTiger

▶ insert_vector_secondary_overall-wiredTiger

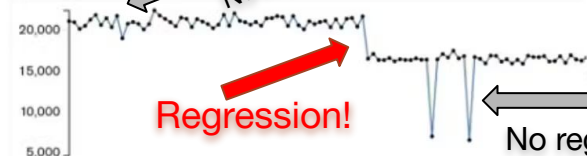
▶ word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298

ops per sec: 5,575

Compare



Regression!

70%
No regression!

VOCABULARY

Signal: The thing we are looking for

Noise: The thing we are not looking for

Outlier: Individual point that is clearly outside of the normal

Anomaly: Outlier

Change: A persistent change in the signal

Variance: Is a specific mathematical variable (which we rarely use)

Variability, range, noise: Use these instead!

EVERYONE ON GITHUB, 2025:

(TURSO DATABASE, GITHUB RUNNERS)

SELECT 1

time ↓

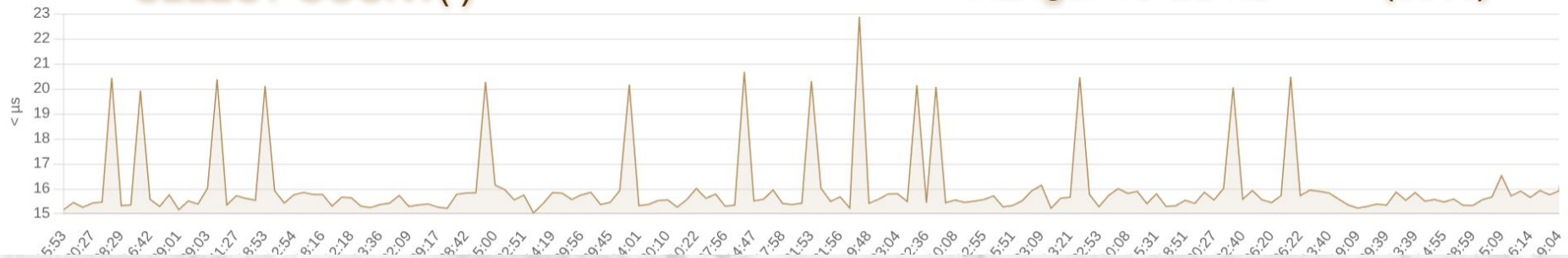
Range: 60-100 ns (40%)



SELECT COUNT(*)

time ↓

Range: 15-23 ns (50%)

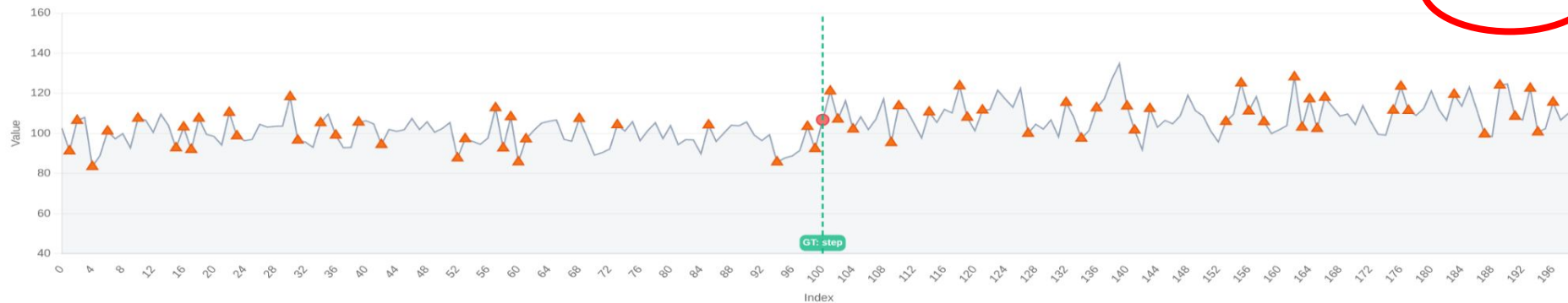


AUTOMATION IS ONE OF THE CORE PRINCIPLES...

Threshold based alerting...

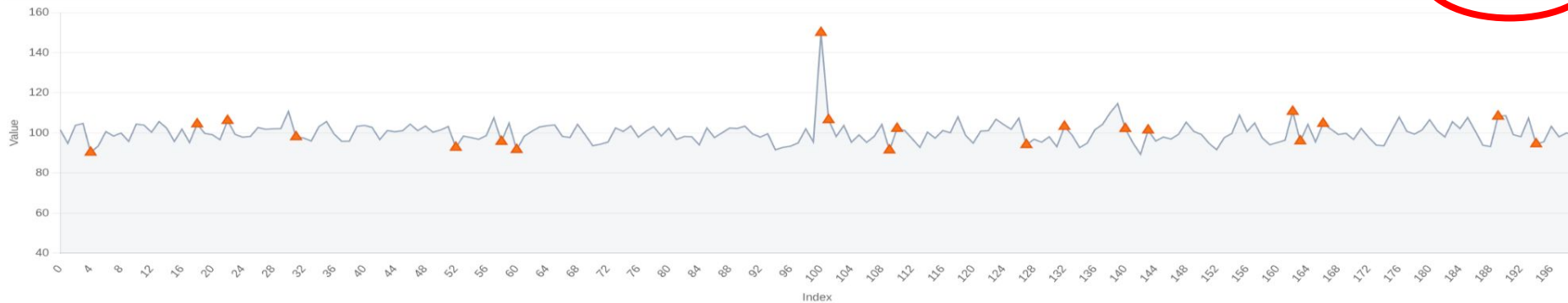
■ Threshold Alert (>9.6%, offset=1)

1 TP / 65 FP



■ Threshold Alert (>9.6%, offset=1)

0 TP / 20 FP



CORE PRINCIPLES (FOR CHANGE POINT DETECTION)

Math is hard*.

*) unfortunately math is also necessary

E-DIVISIVE MEANS (MATTESON & JAMES, 2014)

■ Otava Analysis

1 TP / 0 CM / 0 FP



■ Threshold Alert (>22.1%, offset=1)

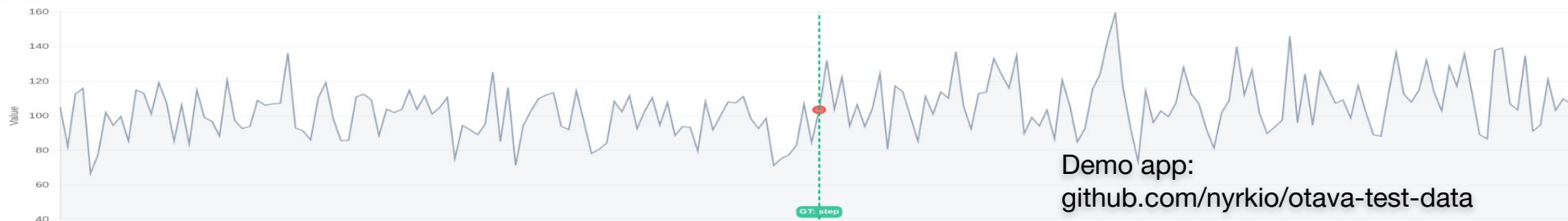
1 TP / 0 CM / 49 FP



A BIT MORE SOPHISTICATED COMPETITION

■ Otava Analysis

1 TP / 0 CM / 0 FP



■ Moving Average Analysis

1 TP / 0 CM / 0 FP



■ Std Dev (M=20, K=3.2σ)

0 TP / 1 CM / 1 FP



OTAVA REALLY SHINES WITH 2 NEARBY POINTS

■ Otava Analysis

2 TP / 0 CM / 0 FP



■ Moving Average Analysis

1 TP / 1 CM / 9 FP



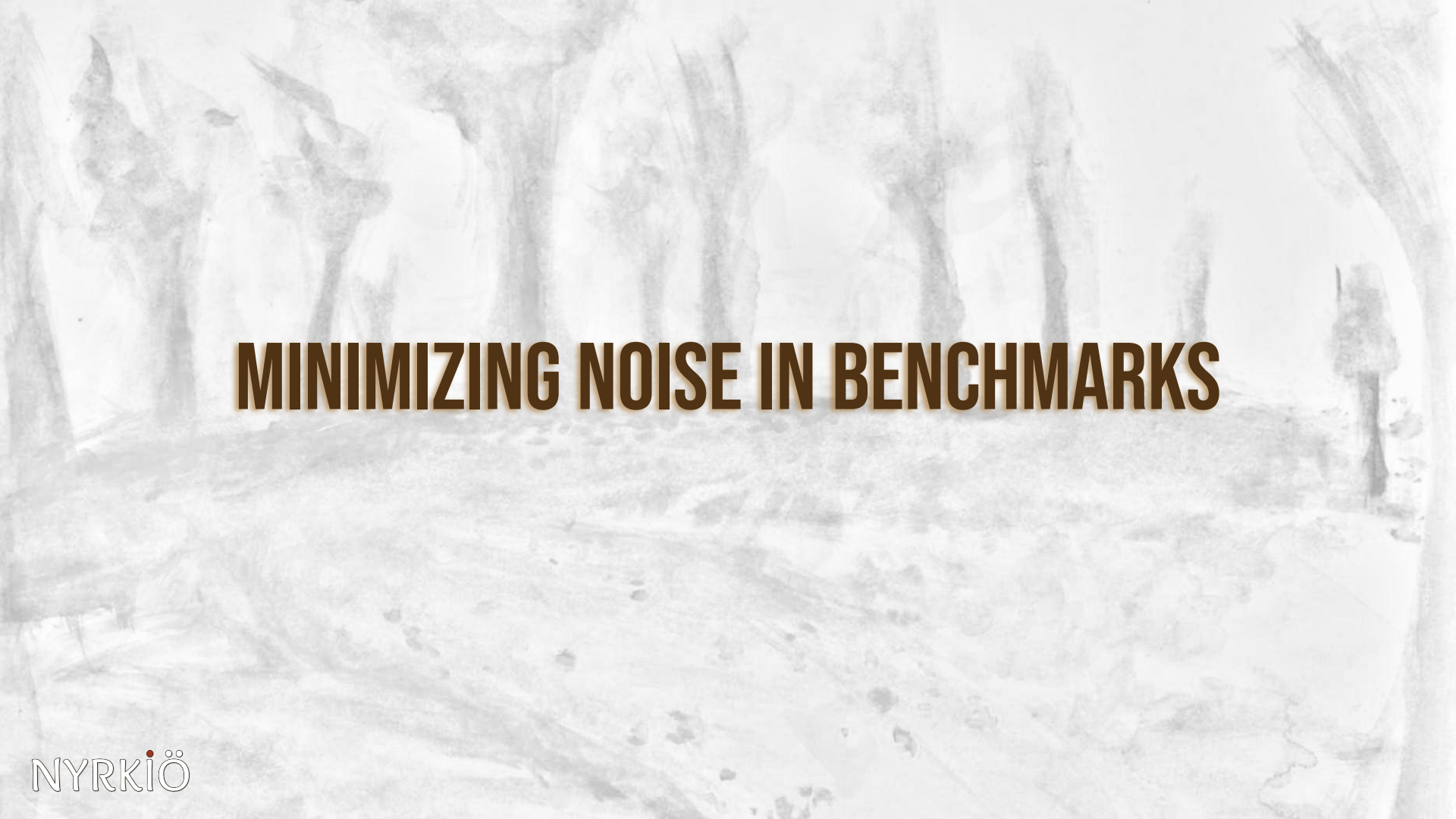
■ Std Dev (M=20, K=2.4σ)

1 TP / 1 CM / 8 FP



A soft, watercolor-style illustration of a forest scene. The background is filled with various shades of grey, brown, and white, creating a misty or ethereal atmosphere. Several tree trunks are visible, some with sparse foliage. The overall texture is painterly and delicate.

otava.apache.org



MINIMIZING NOISE IN BENCHMARKS

CORE PRINCIPLES (FOR SERVER TUNING)

Assume nothing. Measure everything.

Apply the scientific process

Don't believe everything you read on the internet

REASONS WHY BENCHMARK RESULTS ARE SO NOISY?



REASONS WHY BENCHMARK RESULTS ARE SO NOISY?

CLOUD

CLOUD

?

NOISY NEIGHBOR

SOVEREIGN
CLOUD

BAD PROGRAMMERS

FUN EXERCISE: RETROACTIVELY LIST ASSUMPTIONS BUILT INTO YOUR CURRENT ARCHITECTURE

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

For write heavy tests, noise comes from disk

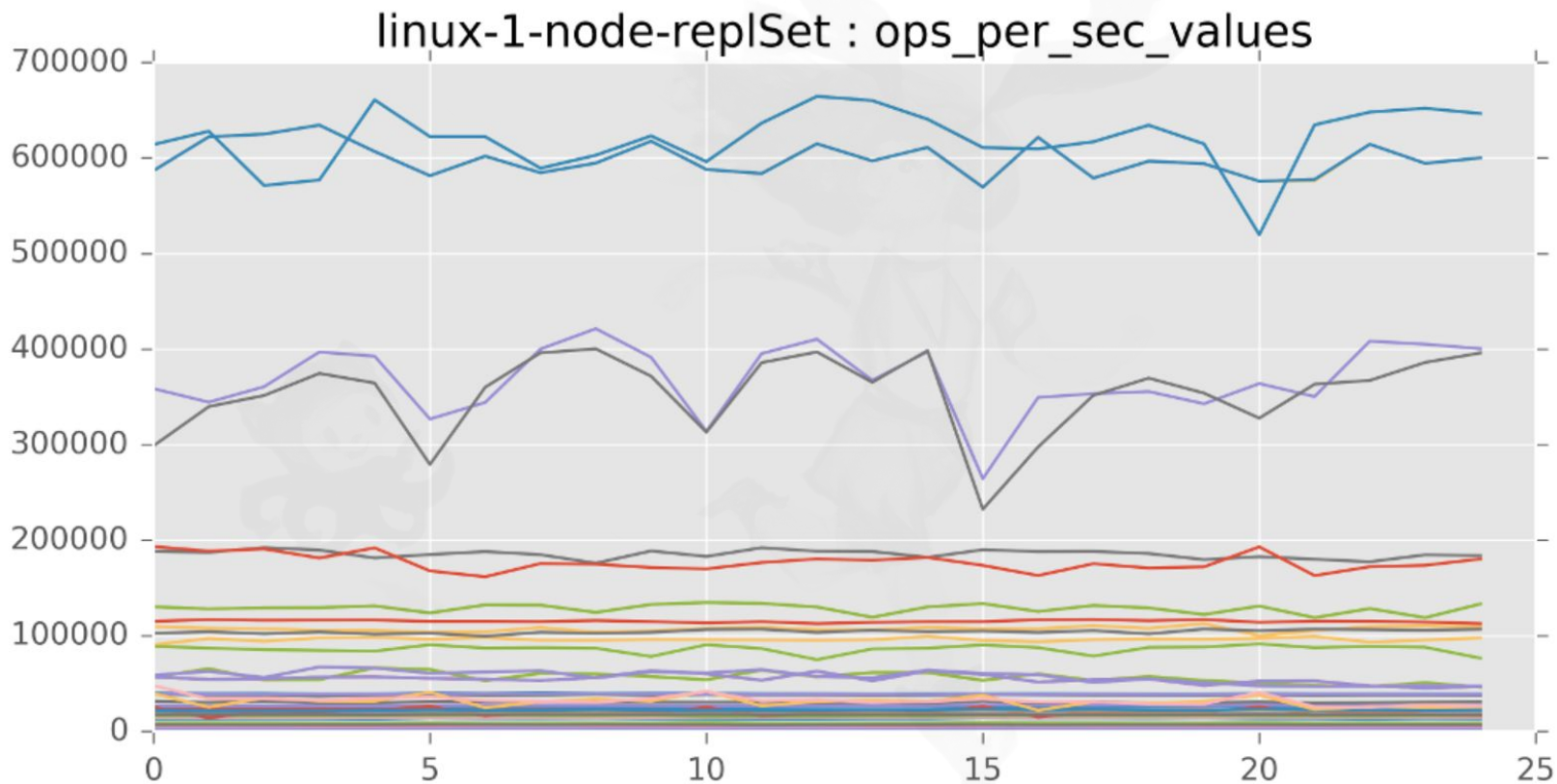
Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances

Just use i2 instances (better SSD)

You can't use cloud for performance testing

1 MONGODB BINARY, 5 SERVERS, REPEAT TESTS 5X (2017)



FUN EXERCISE: RETROACTIVELY LIST ASSUMPTIONS BUILT INTO YOUR CURRENT ARCHITECTURE

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

For write heavy tests, noise comes from disk

Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances

False

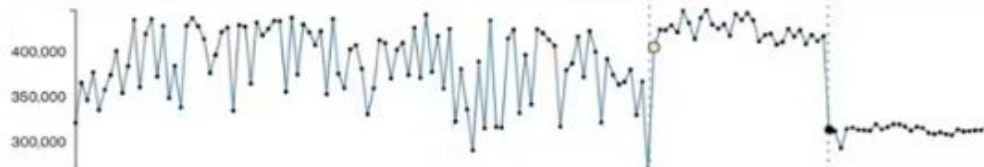
Just use i2 instances (better SSD)

You can't use cloud for performance testing

▼ insert_tti-wiredTiger

c48f298
ops per sec:
313,407

Compare



► insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298
ops per sec:
492,207

Compare



► insert_vector_secondary_load_phase-wiredTiger

► insert_vector_secondary_overall-wiredTiger

► word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298
ops per sec: 5,575

Compare



SSD -> EBS



CPU: No HT, single
socket, scheduling

Noise range for all tests:
5%

WAS IT THE NEIGHBORS?

Continuous Benchmarking is hard because...

- Your hardware is actively working against you:
 - CPU Frequency scaling
 - CPU boost
 - HyperThreading...
 - NUMA architecture...

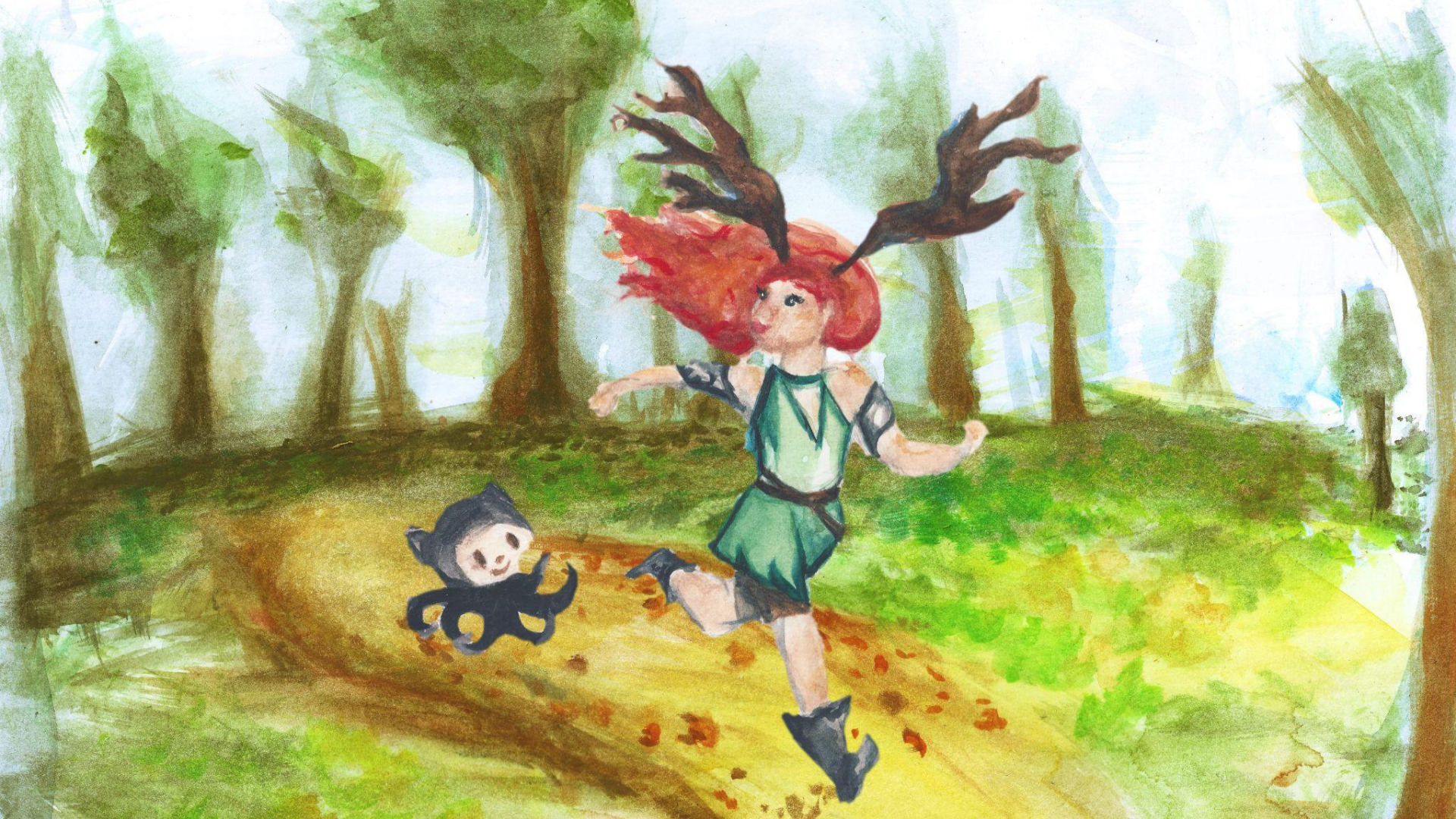
...and that was just the CPU!

```
man cpupower
```




10 years later...

NYRKIÖ



NYRKIÖ

GitHub

Runners



NYRKIÖ

GitHub

Runners



1. Install as GitHub app:
nyrkio.com
2. Pick a subscription (20 c/h)
3. In workflow.yml:

```
runs-on : nyrkio_2
```

C7a instances
Hand picked &
Carefully tuned

NOT best performance
NOT for price/performance

But

REPEATABLE
performance

NYRKIÖ

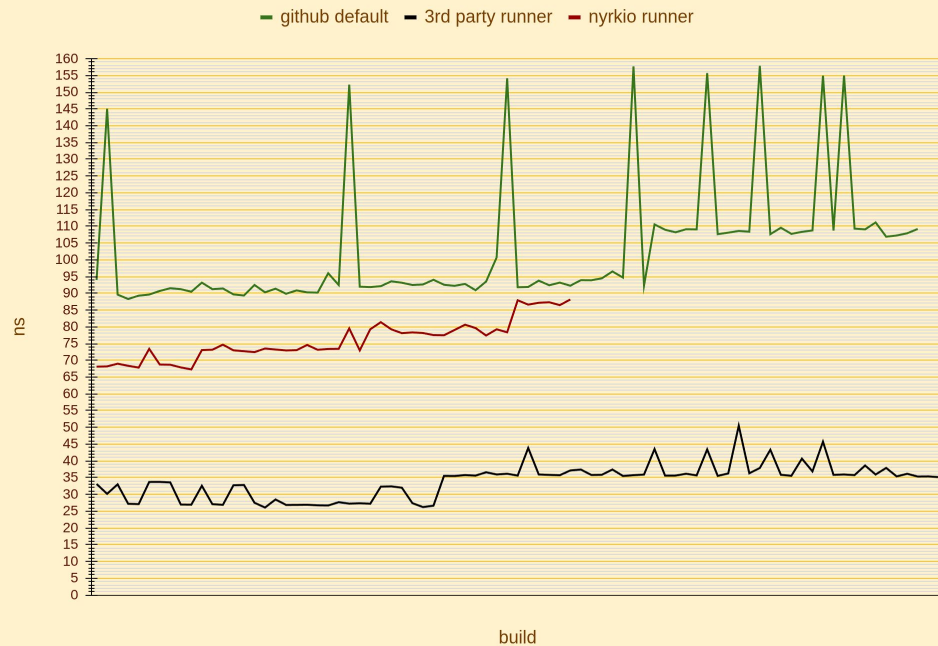
GitHub

Runners

min-max ranges (ns)

github default	3rd party runner	nyrkio runner
88	35	73
154	51	75
66 ns	15 ns	2 ns
75%	44%	3%

SELECT 1



NYRKIÖ

GitHub Runners

min-max ranges (ns)

github default	3rd party runner	nyrkio runner
18	49	11
25	59	13
6 ns	10 ns	1 ns
30%	21%	12%

SELECT COUNT(*)



WHAT IMPROVED IN 10 YEARS?

C7a family offers high fidelity performance for 100% of the price

- No hyperthreading
- Single CPU socket
- AMD?

Future opportunities:

- sub nano-second precision
- Dist-sys: Run K8s cluster, on 128 core server

WHAT HAVE WE LEARNED?

1. Start with a simple benchmark,
Run ***continuously***

3. Performance tuning for *repeatable* results is counter intuitive.

2. Apply

Math & Science

... until .
false positives
no longer hurt

WHAT HAVE WE LEARNED?

1. Start with a simple benchmark,
Run ***continuously***



3. Performance tuning for *repeatable* results is counter intuitive.

2. Apply

Math & Science

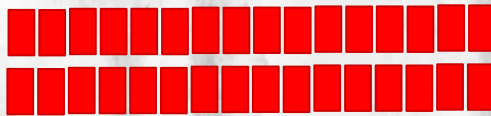
... until
false positives
no longer hurt



PS: CANARIES

Add tests that measure your infrastructure.

- CPU
- Disk
- Network



▼ canary_server-cpuloop-10x

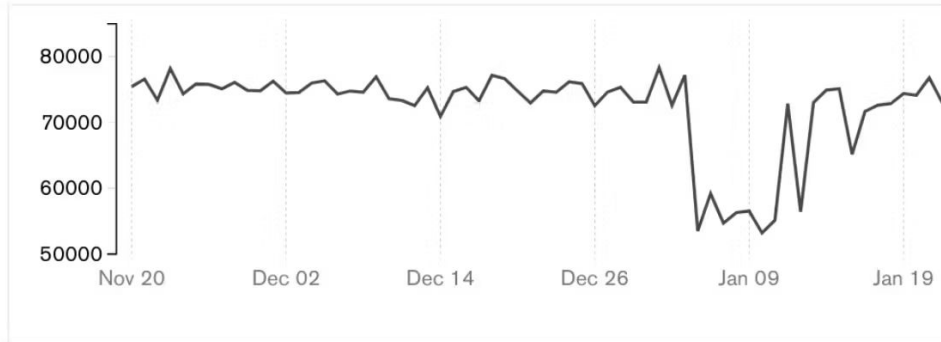
78b96cb

Jan 30 2018

ops per sec:

71,552

[Compare](#)



CREDITS AND REFERENCES

Nyyrikki and cat running, watercolor: Ebba Ingo

Velociraptor: Wikimedia commons

Otava test data graphs:
Joe Drumgoole & Claude

Everyone who contributed to Change Detection,
now known as Apache Otava (incubating)

Ingo,Daly: Reducing Variability in Performance
Tests on EC2

www.youtube.com/watch?v=3kHGZ7niHI4

David Daly et.al.: The Use of Change Point
Detection to Identify Software Performance
Regressions in a Continuous Integration
System, 2020.

Fleming & Kołaczowski: Hunter: Using Change
Point Detection to Hunt for Performance
Regressions, 2023.

Fixing Performance Regressions Before they
Happen (Netflix)

8 Years of Optimizing Apache Otava: How
disconnected open source developers took an
algorithm from n^3 to constant time

Processor state control for Amazon EC2 Linux
instances